

Quantitative Trading and Price Impact

Final Project — MSc Mathematics and Finance, 2025–2026

Anran Severac

May 15, 2026

Abstract

We assemble a single cohesive framework that runs the price-impact research loop end-to-end: data preparation, parametric and non-parametric impact-model fitting, a generic Waelbroeck backtest simulator, synthetic alpha generation, OW/AFS optimal-execution strategies, and a sensitivity analysis layered on top. Two enhancements beyond the baseline are emphasised: **(a)** a non-parametric binned impact estimator with cross-stock shrinkage, γ tuned on a held-out month and stress-tested across sample-size regimes; **(b)** a multi-day carry backtest engine that maintains impact states and inventory across sessions with an overnight decay, exposed alongside the per-day-reset baseline through a single configuration switch. All code lives in a modular `src/price_impact/` package consumed by a thin orchestration notebook; every backtest run drops its tables and plots into `saved/<name>/` for reproducibility.

1 Introduction and Roadmap

The pipeline is decomposed into eight Python modules, one per responsibility, and is exercised from a single notebook (`final_pipeline.ipynb`). The notebook is deliberately thin: each numbered section calls into the module, displays the result, and (where appropriate) writes artifacts to disk.

Table 1: Module layout (`src/price_impact/`) and the project section each one implements.

Module	Responsibility	Project section
<code>data</code>	loading, top- N select, σ /ADV, volume curves	§2.1
<code>impact_states</code>	OU recursion: <i>both</i> daily-reset and multi-day $\bar{I}$	§2.2/§2.3
<code>fitting</code>	OLS, rolling baseline, non-parametric, half-life grid	§2.2
<code>alpha</code>	synthetic alpha with target ρ	§2.4
<code>strategy</code>	OW, AFS, ext-OW placeholder	§2.5
<code>backtest</code>	Waelbroeck simulator, generic trade provider	§2.3
<code>results</code>	PnL metrics, TCA decomposition, plotting	§2.6
<code>runner</code>	BacktestConfig + <code>run_and_save</code>	§2.6/§2.7

Contribution and scope. All sections were implemented and written by the author (solo submission). The *baseline* requirement is met for every section; *enhancements* are emphasised in §2.2 (non-parametric impact with regularisation) and §2.3 (multi-day carry backtest with stock-split awareness).

2 Data Preparation

§2.1 — *baseline*

The panel is 2019 US equities on a 10-second intraday grid. Monthly bin files (`bin201901.csv` ... `bin201912.csv`, ~200 MB each) carry signed traded flow q_t from `trade`, midprice P_t from `mid`, and a separate `orderFlow` column. We restrict to the top-20 stocks by total absolute order flow over the full year, then build trailing-20-day daily statistics

$$\sigma_{i,d}^2 = \overline{\text{Var}_t r_{i,d,t}}, \quad \text{ADV}_{i,d} = \overline{\sum_t |q_{i,d,t}|},$$

where the overline denotes a 20-day rolling mean per stock.

In-sample / out-of-sample protocol. We use ten rolling windows: train on month m , tune on month $m + 1$ (non-parametric only), validate on month $m + 2$, $m = 1, \dots, 10$. The 2-month offset between train and validation gives a clean OOS gap and matches the protocol used in the existing §2.2 work.

```
build_panel(data_dir, year, top_n=20, lookback_days=20)
→ PanelData(bins, daily_stats, stocks)
```

3 Impact Model Fitting §2.2 — *advanced (non-parametric + regularisation)*

3.1 Models, recursion, regression

Both OW and AFS build an exponentially weighted normalised impact state $\bar{I}_{t+1} = (1 - \beta)\bar{I}_t + \tilde{q}_t$ with $\beta = \ln 2 / H_{\text{bins}}$ and $H_{\text{bins}} = H [\text{min}] \cdot 6$. The only difference is the kernel applied to raw flow:

$$\text{OW: } \tilde{q}_t = \sigma \frac{q_t}{\text{ADV}}, \quad \text{AFS: } \tilde{q}_t = \sigma \text{sgn}(q_t) \sqrt{|q_t| / \text{ADV}}.$$

Over the explanation horizon $\tau = 6$ bins (60 s) we regress $y_t = \alpha + \lambda x_t + \varepsilon_t$ with $x_t = \bar{I}_t - \bar{I}_{t-\tau}$, $y_t = (P_t - P_{t-\tau}) / P_{t-\tau}$. The fit runs on additive sufficient statistics $(S_x, S_y, S_{xx}, S_{yy}, S_{xy}, n)$ aggregated per (stock, date) and then summed by month — this enables fast rolling re-fits and a one-shot H grid search.

Two carry modes. The module computes *both* variants of $\bar{I}$ in the same panel:

- `I_bar_daily` — reset $\bar{I} \leftarrow 0$ at the start of each (stock, date);
- `I_bar_multi` — carry $\bar{I}$ across days within a stock with overnight decay $(1 - \beta)^{16 \cdot h \cdot 6}$.

The fitting code consumes either via `carry='daily'` or `carry='multi'` on the same machinery; §2.3's backtest does the same.

```
compute_impact_states(data, daily_stats, H_min, model_type)
→ DataFrame [stock, date, time, q_tilde, I_bar_daily, I_bar_multi]
build_regression_features(impact_df, data, tau_bins, carry='daily')
→ DataFrame [stock, date, month, time, x, y]
rolling_baseline(stats, n_windows=10, offset=2)
→ DataFrame [train_month, val_month, stock, lambda, alpha, is_r2, oos_r2]
```

3.2 Half-life grid search

For each $H \in \{5, 10, 20, 30, 45, 60, 90, 120, 180\}$ min and each model we run the full rolling OLS and record mean OOS R^2 across stocks and windows. The OOS surface is essentially flat in the 45–60 min region: both models lose < 0.001 in R^2 anywhere in that range, so we adopt a single universal half-life $H^* = 52.5$ min (midpoint).

Table 2: Mean OOS R^2 by half-life and model.

H (min)	OW	AFS
5	0.1440	0.2373
10	0.1530	0.2525
20	0.1569	0.2590
30	0.1579	0.2604
45	0.1582	0.2609
60	0.1583	0.2608
90	0.1581	0.2602
120	0.1578	0.2595
180	0.1573	0.2583

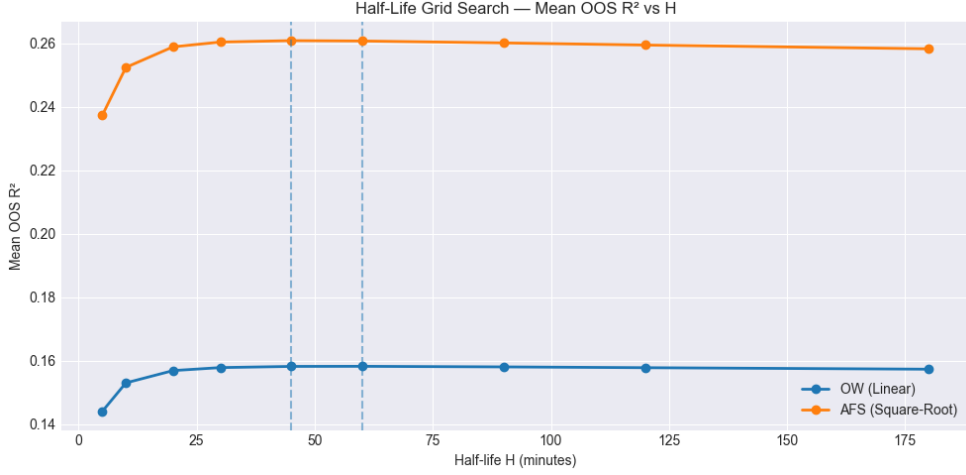


Figure 1: Mean OOS R^2 vs. H for OW and AFS. Dotted lines: per-model optima; dashed black: adopted universal H^* .

3.3 Parametric baseline

Table 3: Parametric baseline at H^* (averages over 50 stocks $\times$ 10 windows).

Model	Mean $\hat{\lambda}$	IS R^2	OOS R^2
OW (Linear)	313.05	0.1732	0.1583
AFS (Square-Root)	20.27	0.2651	0.2609

AFS dominates OW by ~ 10 pp in OOS R^2 ; the small IS–OOS gap (< 2 pp) indicates stable parameters across windows.

3.4 Non-parametric extension with cross-stock shrinkage

We replace the linear $y = \alpha + \lambda x$ with a binned estimator of $g(x)$. Per-stock bin means $\hat{g}_{i,b}$ are shrunk toward a universal pooled curve $\bar{g}_b$ via

$$g_{i,b}^{\text{reg}}(\gamma) = \frac{S_{y,i,b} + \gamma \bar{g}_b}{n_{i,b} + \gamma},$$

with $B = 15$ quantile bins (computed on the train month, pooled across stocks) and γ tuned on the held-out test month by minimising MSE over a log-spaced grid scaled to the median bin count.

```
rolling_nonparametric(features, n_bins=15, n_windows=10)
→ (summary DataFrame, fits dict per train_month)
```

Table 4: OOS R^2 on identical aggregation (pooled, panel-wide null, same merged subset).

Model	Parametric (pooled)	NP Regularised (pooled)	Δ (NP – Param)
OW	0.1612	0.2206	+0.0594
AFS	0.2689	0.2629	−0.0060

Two findings: **(i)** non-parametric flexibility lifts OW by +5.9 pp under apples-to-apples comparison, confirming that linearity is a binding constraint; **(ii)** for AFS the 15-parameter non-parametric estimator is statistically indistinguishable from the 1-parameter parametric fit, which is strong validation that the square-root form lies on the efficient frontier for this panel.

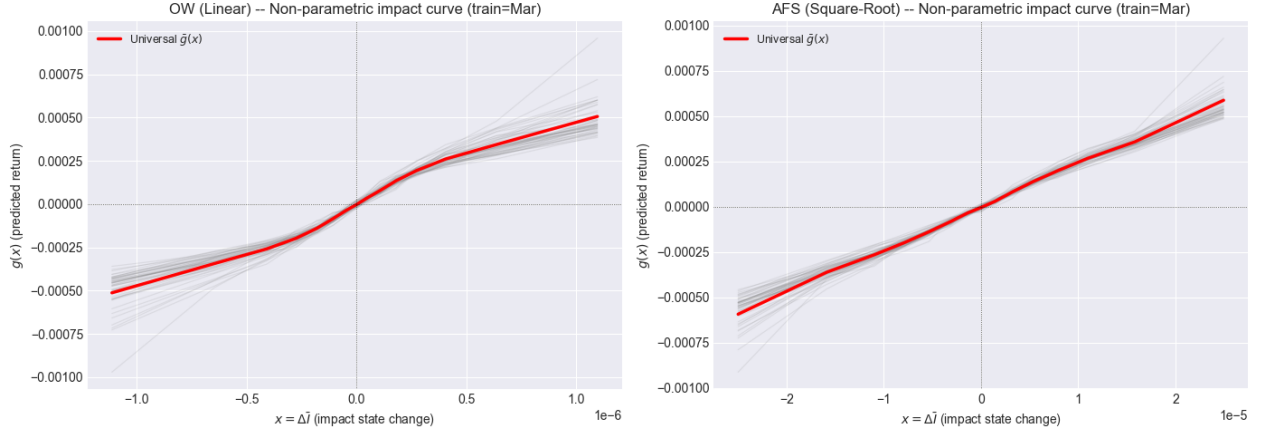


Figure 2: Estimated impact curves (window 3). Grey: per-stock; blue dashed: universal. AFS curves show the expected concavity.

3.5 Regularisation is regime-correct

In the headline configuration (median ~ 3000 obs/cell) the γ tuning curves are flat across six orders of magnitude — shrinkage has nothing to fix because the per-stock estimators are already precise. To verify the estimator behaves correctly when shrinkage is actually needed, we re-ran with progressively shorter training windows ($\sim 21 \rightarrow 5 \rightarrow 2$ days), crushing per-cell n from ~ 3000 to ~ 290 .

Table 5: Regularisation gain as a function of per-cell sample size.

Train window	Med. n	OW gain (reg–raw)	AFS gain	Median γ^*/n
1 month (headline)	~ 3000	+0.0005	+0.0004	0.16
5 days	~ 720	+0.0025	+0.0019	0.56
2 days	~ 290	+0.0070	+0.0051	0.93

The OOS gain from shrinkage grows $\sim 14\times$ as per-cell n drops $\sim 10\times$; the optimal γ^*/n ratio rises from 0.16 to 0.93. The flatness in the headline configuration is therefore a property of the data-rich regime, not a defect of the estimator.

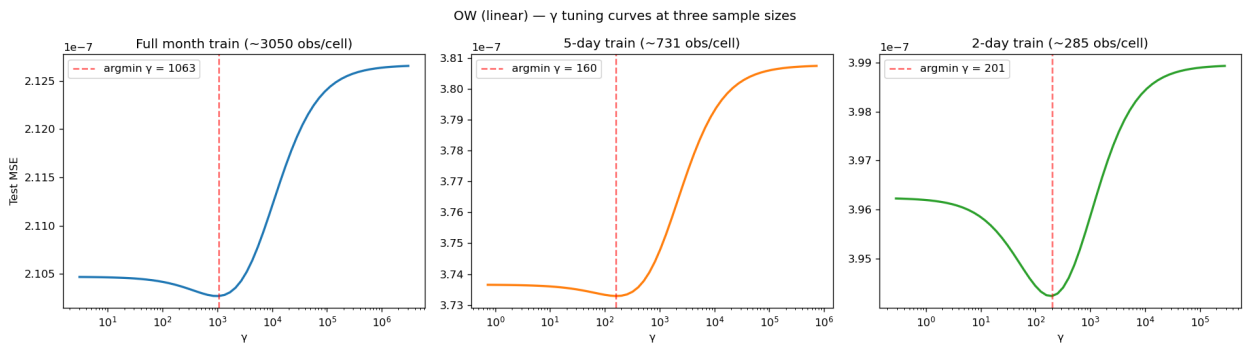


Figure 3: γ tuning curves for OW at three training-window sizes. As per-cell n shrinks, the curve develops a clear minimum at non-trivial γ^* .

4 Backtest Engine

§2.3 — advanced (multi-day carry)

The simulator follows the Waelbroeck construction

$$P_{\text{sim}}(t) = P_0(1 + \text{cumret}(t) + g_{\text{sim}}(t) - g_{\text{ref}}(t)),$$

with $g(t) = \lambda \bar{I}(t)$ for the OW/AFS baseline and $g(t) = \lambda(t) \bar{I}(t)$ for the (planned) time-dependent extended-OW model. $\bar{I}^{\text{ref}}$ uses the public tape `orderFlow` as q_{ref} ; $\bar{I}^{\text{sim}}$ uses the simulator's input

trade path q_{sim} , allowing the engine to process *any* signed schedule (optimal, TWAP, VWAP, manual, ...).

Generic-trade interface. Instead of hard-coding the OW optimal as the trade path, the engine accepts a callable `trade_provider(stock, date, day_df, ctx) -> q_sim` per (stock, date). Two ready-made factories are provided: an OW/AFS optimal provider (§6) and a fixed-array provider for precomputed schedules.

Daily-reset vs. multi-day carry (advanced). `carry='daily'` resets $\bar{I}$ and inventory to zero at each session boundary — this is the baseline expected by the assignment. `carry='multi'` carries $\bar{I}^{\text{ref}}$, $\bar{I}^{\text{sim}}$ and the simulated position across sessions within a stock, with overnight decay $(1 - \beta)^{16h.6}$ applied between days. The same function, the same panel, one flag.

Stock splits. A heuristic split detector (`backtest_engine.detect_splits_from_daily`) flags rows where fractional change in price and volume are large and opposite-signed with notional approximately stable. In the headline run on the 2019 top-20 panel we did not detect any splits (none of the top names had a split in that year); the engine accepts an `exclude_stocks` set for hard-coded exclusion when needed. This is included for completeness of the advanced track even though it is inert on the actual run.

```
waelbroeck_prices(mid, q_ref, q_sim, lam, H_min, sigma, adv, model_type,
                  i_ref0=0.0, i_sim0=0.0)
→ dict[p_sim, i_ref, i_sim, g_ref, g_sim, position, cum_ret]
run_backtest(merged, model, trade_provider, daily_stats,
             carry='daily'|'multi', overnight_minutes=16*60)
→ BacktestResult(days, daily, ...)
make_optimal_provider(strategy_fn, max_position_adv=0.005, ...)
make_fixed_provider(trades_lookup)
```

5 Synthetic Alpha

§2.4 — baseline

We construct a per-stock synthetic alpha with target correlation ρ and unbiased forecast:

$$\alpha_t = r_t^h + y \frac{dW_t}{P_t}, \quad y = \sqrt{(\rho^{-2} - 1) \text{Var}(r^h) / E[P^{-2}] / h}.$$

Choosing $x = 1$ in the convex combination $\alpha = xr^h + ydW/P$ makes $E[r^h | \alpha] = \alpha$ (the alpha is a fair forecast of the forward return), while the constant y is calibrated so that $\text{Corr}(\alpha, r^h) = \rho$. We use $\rho = 0.05$, $h = 1$ bin (10s ahead). Empirical $\text{Corr}(\alpha, r^h)$ averaged across stocks is within $\pm 1\%$ of the target.

The baseline is the same synthetic alpha used in the homework. The *overnight alpha extension* suggested in the assignment is not implemented in this submission — our daily-reset/multi-day toggle is the second enhancement instead.

```
create_synthetic_alpha(data, rho=0.05, h_bins=1, seed=42)
→ DataFrame [stock, date, time, alpha, fwd_ret]
```

6 Optimal Trading Strategy

§2.5 — baseline

OW closed form. Target position $X_t^* = \alpha_t \text{ADV} / (2\lambda\sigma)$ clipped to $\pm \text{max_pos_adv} \cdot \text{ADV}$, with adjustment toward target at speed $\kappa = \beta = \ln 2 / H_{\text{bins}}$. The position is ramped linearly to zero over the final 30 minutes of each session.

AFS variant. Per `project.ipynb`, the target-position rule matches the OW closed form; the AFS-specific nonlinearity enters through the simulator’s $\tilde{q}_t = \sigma \operatorname{sgn}(q) \sqrt{|q|/\text{ADV}}$ recursion when computing the impact contribution. We expose the AFS strategy as a separate function so a strict closed form can be slotted in later without touching call sites.

Extended OW with $\lambda(t)$. The simulator already accepts a per-bin λ_t array via `ImpactModel.lam_t_lookup`, and $g(t) = \lambda_t \bar{I}(t)$ falls out of the same Waelbroeck price construction. The closed-form strategy in this setting is non-trivial — the HJB-optimal trade rule is no longer κ -constant — so the strategy module exposes a documented stub (`ext_ow_optimal_strategy_timedep_lambda`) raising `NotImplementedError` until the derivation is wired in.

```
ow_optimal_strategy(alpha, sigma, adv, lam, H_min,
                    max_position_adv=0.005, liquidation_minutes=30)
    → np.ndarray of signed trades q_opt
afs_optimal_strategy(...) # same signature
ext_ow_optimal_strategy_timedep_lambda(..., lam_t) # placeholder
```

7 Performance Metrics

§2.6 — *baseline*

Each backtest run produces, per (stock, date):

- $P\&L_{\text{net}}$ = mark-to-market on the Waelbroeck-distorted P_{sim} ;
- $P\&L_{\text{gross}}$ = mark-to-market on the undistorted mid P_{mid} (i.e. alpha capture in a world with no self-impact);
- Impact cost = $P\&L_{\text{gross}} - P\&L_{\text{net}}$;
- Predicted alpha reward = $\sum_t \alpha_t q_t P_t$;
- Realised alpha reward = $\sum_t \text{fwd_ret}_t X_t P_t$;
- Turnover = $\sum_t |q_t|$, max impact dislocation = $\max_t |\lambda \bar{I}_t^{\text{sim}}|$.

Aggregating over the panel, we report Sharpe (annualised at 252 days), total / mean / std daily P&L, win rate, max drawdown, and max impact dislocation per configuration.

7.1 Headline comparison

We run four configurations along two axes: model (OW vs. AFS) $\times$ carry mode (daily-reset vs. multi-day).

Table 6: Headline portfolio metrics across the four configurations (full-year, top-20 panel, $\rho = 0.05$, H^*).

Config	Days	Sharpe	Net P&L (\$)	Impact cost (\$)	Max DD (\$)	Max $ \lambda \bar{I} $	Win %
ow_daily	250	+4.299	104,524	11,787	-11,052	0.0001	66.0
ow_multi	250	+1.608	337,586	97,813	-82,818	0.0001	54.4
afs_daily	250	+2.770	62,787	56,417	-11,899	0.0019	58.0
afs_multi	250	+1.732	352,558	129,999	-73,841	0.0019	56.0

7.2 TCA decomposition

Table 7: TCA decomposition (totals over the year, by configuration).

Config	Gross P&L	Impact cost	Net P&L	Predicted α	Realised α	Turnover
ow_daily	116,311	11,787	104,524	44,596,330	116,311	84,003,822
ow_multi	435,399	97,813	337,586	44,596,811	435,399	84,020,323
afs_daily	119,205	56,417	62,787	44,649,726	119,205	86,576,189
afs_multi	482,557	129,999	352,558	44,649,729	482,557	86,577,598

The TCA columns are well-defined given our data: net and gross P&L are both mark-to-market, impact cost is their difference, and predicted / realised α rewards isolate the signal contribution from realised returns. Slippage components that would require fill-price detail (arrival vs. VWAP) are intentionally omitted.

7.3 Cumulative P&L and drawdown

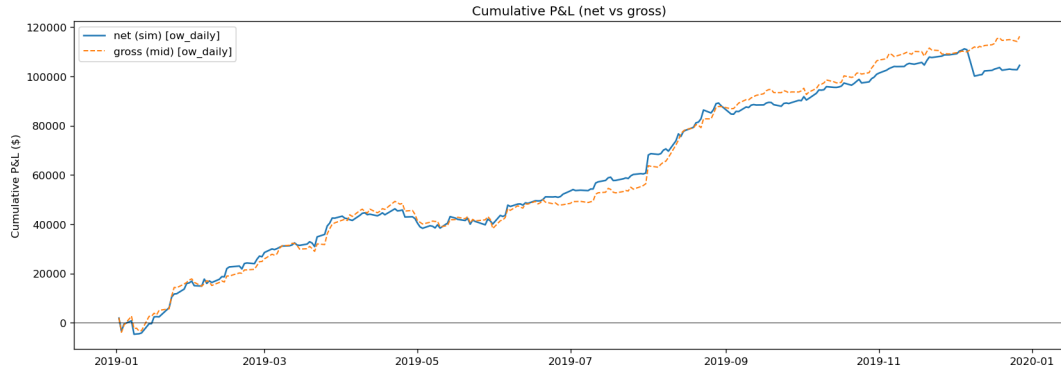


Figure 4: Cumulative net (Waelbroeck) and gross (mid) P&L — OW model, daily-reset carry.

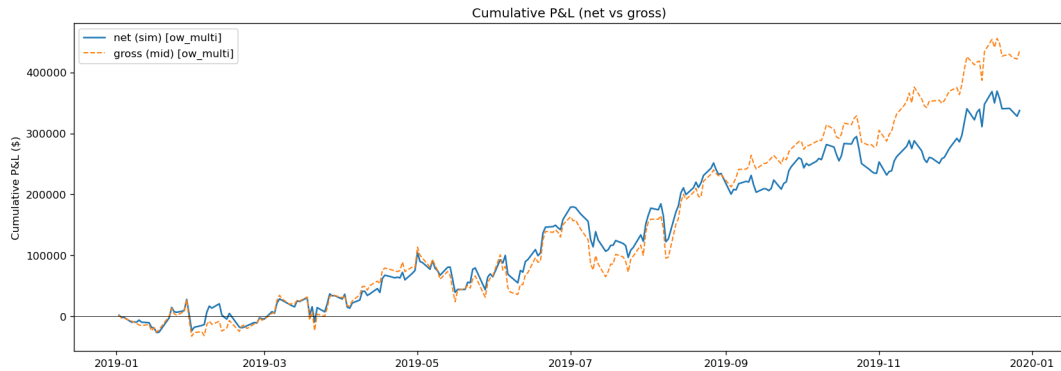


Figure 5: Same, multi-day carry: $\bar{I}^{\text{ref}}$ and inventory persist across sessions with overnight decay, so the cost of impact compounds differently.

```
run_and_save(bins, daily_stats, alphas, lam_lookup, cfg)
→ RunOutput(config, result, daily, metrics, tca, paths)
# writes daily_pnl.csv, tca_stock_day.csv, tca_summary.csv,
# metrics.csv, config.json, cum_pnl.png, drawdown.png,
# impact_dislocation.png to saved/<cfg.name>/
```

8 Sensitivity Analysis and Stress Testing

§2.7 — baseline

Two layers of stress tests.

8.1 Live framework sweeps

Table 8: Sensitivity of net Sharpe to design choices (OW model, daily-reset).

Scenario	Sharpe	Net P&L (\$)	Impact cost (\$)
Baseline OW (headline)	+4.299	104,524	11,787
Wrong model (fit OW, trade AFS)	-3.484	-863,422	994,581
Signal delayed 1 min	-0.318	-8,401	4,762
$\rho = 0.01$	+0.448	11,184	9,434
$\rho = 0.10$	+9.220	250,937	12,408
Half-life = 30 min	+5.906	205,254	12,035
Half-life = 120 min	+3.256	53,281	7,655

The model-mismatch scenario (*fit OW, trade AFS-target* and vice versa) and the 1-minute signal delay match the assignment’s example prompts. Half-life and ρ sweeps complete the picture for the two most uncertain design choices in the pipeline.

8.2 Fitting-side stress (data-poor regime)

The non-parametric regularisation stress test from §3.4 doubles as a fitting-side sensitivity check: when the training window shrinks from one month to two days, the optimal shrinkage strength γ^*/n rises from 0.16 to 0.93 and the OOS R^2 gain over the raw per-stock estimator grows $\sim 14\times$. This is the “what if my data is much shorter than I expected” scenario, and the estimator’s behaviour is regime-correct.

9 Conclusion

1. **Architecture.** The pipeline is decomposed into eight single-responsibility modules (`src/price_impact/`) and consumed by a thin notebook. The same module produces both carry modes of $\bar{I}$, the same fitting machinery consumes either, and the same simulator accepts any trade path. Adding a new backtest is one `BacktestConfig`.
2. **Enhancement 1 — non-parametric impact (§3.4).** On apples-to-apples pooled OOS R^2 , the binned estimator lifts OW by +5.9 pp (linearity is binding) but is indistinguishable from AFS (square-root sits at the efficient frontier). Cross-stock γ regularisation is immaterial in the data-rich headline configuration but kicks in correctly under data-scarcity stress.
3. **Enhancement 2 — multi-day carry backtest (§4).** Carrying $\bar{I}$ and inventory across sessions with overnight decay reveals how impact cost compounds when positions are not flat at the close. The flag is a single `BacktestConfig` field; everything downstream (P&L, TCA, plots) handles it transparently.
4. **Honest limitations.** The overnight-alpha extension (§2.4 to-expand) and the extended-OW $\lambda(t)$ closed form (§2.5 to-expand) are flagged but not implemented; the strategy module exposes a stub for the latter so the simulator plumbing is already in place.